CHEAT SHEET · JS INTERVIEW #17 @specsycoder

Promise vs setTimeout

Why it prints 1, 4, 3, 2

order.js

```
1  console.log("1");
2  setTimeout(() => console.log("2"), 0);
3  Promise.resolve().then(() => console.log("3"));
4  console.log("4");
```

output 1 4 3 2

Why it happens

01 Sync code first

Normal lines run right away: 1, then 4.

02 Two queues

setTimeout goes to the task queue. .then goes to the microtask queue.

03 Microtasks win

When the stack is empty, ALL microtasks run before any task.

04 0ms is not instant

The timer still waits for sync code and every microtask.

`sync code → ALL microtasks → one task → ALL microtasks → …`

Save this · share with a friend

Follow @specsycoder for daily JS interview Qs

The await follow-up

Code after await is a microtask

async-await.js

```
1  console.log("A");
2  setTimeout(() => console.log("B"));
3  (async () => {
4    console.log("C");
5    await null;
6    console.log("D"); // runs as a microtask
7  })();
8  console.log("E"); // output: A C E D B
```

microtasks (first)	tasks (after)
.then / .catch / .finally	setTimeout, setInterval
code after await	click and other UI events
queueMicrotask()	network callbacks
MutationObserver	MessageChannel

Traps to mention

- **The code before await runs synchronously.** That is why "C" prints before "E".
- **Endless microtasks block the page.** Timers and clicks never get a turn.

Your 20-second answer

"Sync code runs first, so 1 and 4. Then the event loop empties the microtask queue, so the promise prints 3. Only after that does it take a task, so the setTimeout prints 2, even with 0ms. Promises and code after await are microtasks; timers and events are tasks."